Software Architecture

EE 8217/EC 8208

Based on Software Architecture in Practice (3rd Edition)

By Len Bass, Paul Clements, Rick Kazman

Outline

Introduction for the module

- What is Software Architecture?
- Why Architecture Matters?
- Why Do We Need Software Architecture?
- What Will You Learn in This Module?

What is Software Architecture?

- Software Architecture defines **fundamental organization** of a system and more simply defines a structured solution.

- It determines how the various components of a software system are assembled, how they relate to one another, and how they communicate. Essentially.

- it serves as a **blueprint** for the application and a foundation for the **development team** to build upon.

What is Software Architecture?

The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.

Software architecture defines:

- **System structure**
- **System behavior**
- **Component relationships**
- **Communication structure**
- **Stakeholder balance**
- **Team structure**
- **Early design decisions**

Architecture Is a Set of Software Structures

- A structure is simply a set of elements held together by a relation.
- Software systems are composed of many structures, and no single structure holds claim to being the architecture.
- There are three categories of architectural structures, which will play an important role in the design, documentation, and analysis of architectures:

- Architecture deals with
  - Abstraction
  - Decomposition and composition
  - Styles

Every Software System Has a Software Architecture

- Any system consists of **elements and relationships**, which form its architecture.

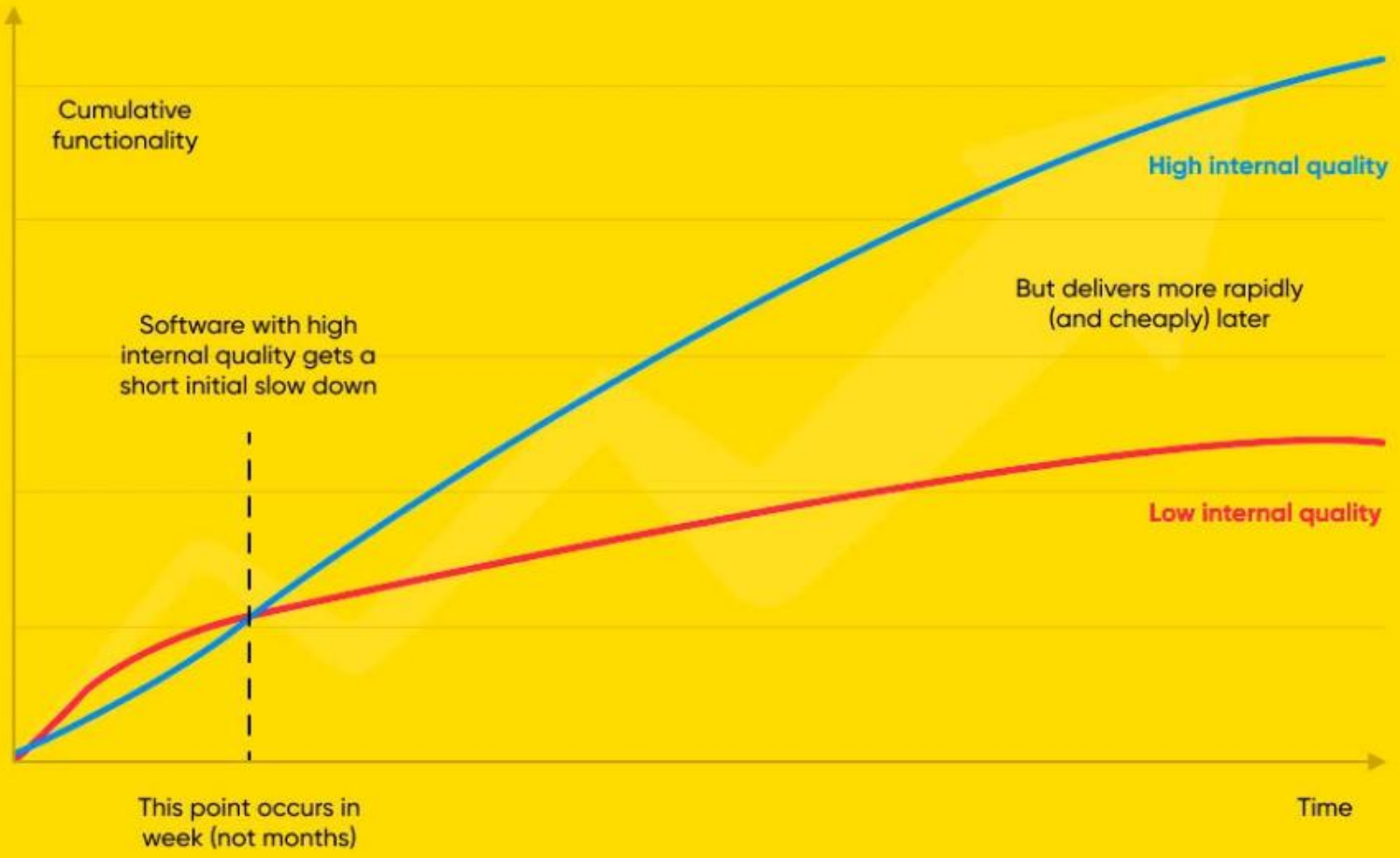
- Even a very simple system (just one element) still has an architecture.

- However, the architecture may **not be known or documented**
  - Designers may be gone
  - Documentation may be missing
  - Only the running system may exist

Why Architecture Matters?

- Impacts performance, scalability, security
- Difficult to change later
- Foundation for development
- Represents earliest design decisions
- Aids in communication with stakeholders
- Defines constraints on implementation
- Supports predicting cost, quality, and schedule
- Aids in software evolution

Difference between low quality high quality software



Why Do We Need Software Architecture?

- Modern Engineering Systems Are Software-Driven

  - Smart devices
  - Power systems
  - Mobile applications
  - Embedded systems
  - Robotics & automation
  - Cloud and IoT systems

Software Architecture helps engineers to:

- Design systems before implementation
- Manage complexity in large projects
- Improve:
  - Reliability
  - Scalability
  - Security
  - Performance
- Reduce future maintenance problems

What Will You Learn in This Module?

- # Introduction to Software Architecture

  - What software architecture is ?
  - Importance of architectural decisions ?
  - Architectural descriptions and views ?

Software Design Concepts

- Abstraction
- Modularity
- Separation of concerns
- Information hiding
- Functional independence

“Smart University Management System”

The university wants to develop a system with:

- Student registration
- Attendance management
- Online payments
- Medical services
- Library management

Design Models

- **Data design** (ER diagrams Database schema)
- **Interface design** (UI, System Interfaces)
- **Component-level design** (internal design of software components/modules.)
- **Deployment design** (Where does the software run?)

Architectural Styles

- **Layered architecture**
- **Client–Server architecture**
- **Object-Oriented architecture**
- **Data-flow architecture**
- **etc….**

Component-Level Design

- Designing software components
- Transforming requirements into designs.
- It describes:
  - What each component does
  - How components interact
  - The functions, methods, and logic inside a component

Design Patterns

- Design patterns are reusable and proven solutions to common software design problems.
- They are not complete programs, but **templates or best practices** used during software design.

Why Do We Need Design Patterns?

- Without proper design:
  - Code becomes difficult to maintain
  - Similar problems are solved repeatedly
  - Systems become tightly coupled
- Design patterns help developers:
  - Reuse proven solutions
  - Improve code quality
  - Increase maintainability
  - Reduce development time

Think about building construction

Architects reuse common designs for doors, windows, stairs

Similarly:

Software engineers reuse design patterns for common software problems

Learning Objectives

At the end of the module:

You will be able to:

- LO-1 Explain the design concepts of Software Engineering
- LO-2 Identify the elements of a software design model
- LO-3 Describe software architectural styles
- LO-4 Conduct component level design
- LO-5 Apply design patterns to software design problems

Credits : 2

Total Lectures (Hr) : 26

Methods of Assessments and Marks Allocation

Continuous Assessments 50%

a) 2 x Software Assignment [20%]

b) 1 x Project [30%]

End Semester Examination 50%

a) Written Examination [50%]

Software Assignment

- Analyze the architecture of an existing real-world system.
- Apply software design concepts to a problem scenario.

Same Scenario…

Project

Architecture-Centered System Design Project

- Phase 1 — Requirement & Architecture Design
- Phase 2 — Component-Level Design
- Phase 3 — Prototype / Partial Implementation
  - Small prototype is enough
  - Focus should remain on architecture and design